Fun with Functions

Functions are simple at first glance—they have a name, arguments, and a body. But Elixir allows you to overload functions in several different ways that aren't possible in most other languages. Even though they are simple, this ability to overload them makes functions very powerful.

You wrote named functions in the previous section. In this section, you'll deal with anonymous functions. Anonymous functions are a quick and convenient way to define functions inline in your code.

Let's get started with anonymous functions.

Anonymous Functions

Anonymous functions are ones that have no name. They are commonly used when passing behavior into a different function, such as an enumeration function.

Anonymous functions are quick and easy to define, and they keep behavior directly next to the call site. These attributes make them convenient to write and easy to read. However, the biggest benefit of them is that scoping is maintained. All of the variables that are in-scope where the anonymous function is defined can be used by the anonymous function.

Here's an example of anonymous functions. Type the following into IEx:

```
$ iex
iex(1)> Enum.map([1, 2, 3], fn num -> num * 2 end)
[2, 4, 6]

iex(2)> iterate = fn num -> num * 2 end
#Function<44.40011524/1 in :erl_eval.expr/5>

iex(3)> Enum.map([1, 2, 3], iterate)
[2, 4, 6]

iex(4)> iterate.(2)
4
```

Anonymous functions are similar to Procs in Ruby. Let's look at Ruby code that uses Procs to enumerate over an array.

```
$ irb
irb(main)> [1, 2, 3].map { |num| num * 2 }
=> [2, 4, 6]

irb(main)> iterate = ->(num) { num * 2 }
=> #<Proc:0x000000014422e918 (irb):1 (lambda)>
```